微算機實習**Lab10**

組別:第**14**組

資工三 **112590034** 高翎竣
資工三 **112590032** 陳俊彥
資工三 **112590453** 詹東儒

日期:**114**年**12**月**26**日

一、實驗內容

將Lab06+Lab09的項目延伸加入物聯網，並且加入光敏電阻，如果夠亮會顯示進行運算，太暗會暫

Thread 角色定義：

- Thread A (Sensor Node): 負責讀取 ADC (光敏電阻)，並更新全域環境數值。
- Thread B (Worker Node): 負責核心運算 (Lab9 功能) 與 LED 控制 (依據環 境數值決定是否工作)。
- Thread C (Web Server): 負責提供 GUI，顯示目前的環境數值與運算狀態。停運算，運算完即停止。

二、實驗過程及結果

I. 實驗過程：

先將Lab06跟Lab09重新複習一遍後，將我們需要的部分程式做改造，包括Lab06用python寫的光敏電阻測試程式以及Lab09做的內容，並依照項目要求作呈現。

II. 實驗結果：

數值會顯示、燈會依照光線的明亮閃爍快或慢，當過暗就暫停計算，足夠亮後繼續計算，直到結束停止計算，並且正確使用pthread_mutex_lock。

網址：https://youtu.be/am6nQgBPNb4

III. 遇到的問題以及怎麼解決：

我們原本程式都執行的很順利，”總體來說”沒什麼問題，但是原本我們是要用Node.js做靜態網頁的，結果在詢問LLM的過程中它把HTML做在Lab10.cpp裡而且接近我們的需求，感覺挺酷挺新穎的，於是我們就把它留下來了，但也成為我們的負擔，因為Lab10就變得很長很難debug，我們得先搞清楚它在寫什麼才有辦法改它，不過修改後結果是好的，而且Lab10裡也沒有硬性規定一定要用Node.js。

三、程式碼

Makefile:

```
obj-m := demo.o

kernel_dir := /usr/src/linux-headers-4.9.201-tegra-ubuntu18.04_aarch64/kernel-4.9

PWD := $(shell pwd)

all:
	make -C $(kernel_dir) SUBDIRS=$(PWD)
clean:
	rm -f *.o *.ko *.mod.c
.PHONY:
	clean
```

demo.c:

```
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/fs.h>
#include <linux/string.h>
#include <linux/uaccess.h>
#include <asm/uaccess.h>

#define MAJOR_NUM 60
#define MODULE_NAME "demo"

static int iCount = 0;
static char userChar[100];

//  LED status
static char led_status[4] = {'0', '0', '0', '0'}; //  LED1 ~ LED4

// Open function
static int drv_open(struct inode *inode, struct file *filp) {
	printk("Enter Open function\n"); // [cite: 303]
	return 0;
}

// Release function
static int drv_release(struct inode *inode, struct file *filp) {
	printk("Enter Release function\n"); // [cite: 304]
	return 0;
}
```

```c
static void file_write(char *path, char *str) {
    struct file *fp;
    loff_t pos = 0;
    mm_segment_t old_fs;

    old_fs = get_fs();
    set_fs(get_ds());

    fp = filp_open(path, O_WRONLY, 0);
    if (IS_ERR(fp)) {
        printk("File open error: %s\n", path);
    } else {
        vfs_write(fp, str, strlen(str), &pos);
        filp_close(fp, NULL);
    }

    set_fs(old_fs);
}

// Read function : return LED status to User
static ssize_t drv_read(struct file *filp, char *buf, size_t count, loff_t *ppos) {
    printk("Enter Read function\n");

    // [Fix] let led_status copy to user's buf
    // return recorded 4 LED status for real time
    int ret;
    if (count < 4) return -EINVAL; // Buffer 2 small

    ret = copy_to_user(buf, led_status, 4);
    if (ret != 0) {
        return -EFAULT;
    }

    return 4; // return loaded 4 bytes
}
```

```c
// Write function：receive User operation then control LED
static ssize_t drv_write(struct file *filp, const char *buf, size_t count, loff_t *ppos) {
        printk("device write\n");
        printk("%d\n", iCount);
        printk("W_buf_size: %d\n", (int)count);
        if(count > 99){ count = 99; }   //

        copy_from_user(userChar, buf, count);
        userChar[count] = '\0';
        printk("userChar(char): %s\n", userChar);

        char *gpio_path_led1 = "/sys/class/gpio/gpio396/value";         //pin 7
        char *gpio_path_led2 = "/sys/class/gpio/gpio466/value";         //pin 11S
        char *gpio_path_led3 = "/sys/class/gpio/gpio297/value";         //pin 32
        char *gpio_path_led4 = "/sys/class/gpio/gpio254/value";         //pin 22

        if (strncmp(userChar, "LED1on", 6) == 0) {
                file_write(gpio_path_led1, "1");
                led_status[0] = '1';
                printk("LED1 turned ON\n");
        } else if (strncmp(userChar, "LED1off", 7) == 0) {
                file_write(gpio_path_led1, "0");
                led_status[0] = '0';
                printk("LED1 turned OFF\n");
        } else if (strncmp(userChar, "LED2on", 6) == 0) {
                file_write(gpio_path_led2, "1");
                led_status[1] = '1';
                printk("LED2 turned ON\n");
        } else if (strncmp(userChar, "LED2off", 7) == 0) {
                file_write(gpio_path_led2, "0");
                led_status[1] = '0';
                printk("LED2 turned OFF\n");
        } else if (strncmp(userChar, "LED3on", 6) == 0) {
                file_write(gpio_path_led3, "1");
                led_status[2] = '1';
                printk("LED3 turned ON\n");


        } else if (strncmp(userChar, "LED3off", 7) == 0) {
                file_write(gpio_path_led3, "0");
                led_status[2] = '0';
                printk("LED3 turned OFF\n");
        } else if (strncmp(userChar, "LED4on", 6) == 0) {
                file_write(gpio_path_led4, "1");
                led_status[3] = '1';
                printk("LED4 turned ON\n");
        } else if (strncmp(userChar, "LED4off", 7) == 0) {
                file_write(gpio_path_led4, "0");
                led_status[3] = '0';
                printk("LED4 turned OFF\n");
        }

        iCount++;
        return count;
}

// I/O Control function
static long drv_ioctl(struct file *filp, unsigned int cmd, unsigned long arg) {
        printk("Enter I/O Control function\n"); // [cite: 307]
        return 0;
}

// define file_operations [cite: 1013]
struct file_operations drv_fops = {
        .read = drv_read,
        .write = drv_write,
        .unlocked_ioctl = drv_ioctl,
        .open = drv_open,
        .release = drv_release,
};
```

```c
// Init function [cite: 1028]
static int demo_init(void) {
        if (register_chrdev(MAJOR_NUM, MODULE_NAME, &drv_fops) < 0) {
                printk("%s: can't get major %d\n", MODULE_NAME, MAJOR_NUM);
                return -EBUSY;
        }
        printk("<1>%s: started\n", MODULE_NAME);
        return 0;
}

// Exit function [cite: 1040]
static void demo_exit(void) {
    unregister_chrdev(MAJOR_NUM, MODULE_NAME);
    printk("<1>%s: removed\n", MODULE_NAME);
}

module_init(demo_init);
module_exit(demo_exit);
MODULE_LICENSE("GPL");
```

read_adc.py:

```python
import Jetson.GPIO as GPIO
import time
import sys

# SPI pins (BOARD numbering - Physical pin numbers)
SPICLK = 23   # Board pin 23 (BCM 11)
SPIMISO = 21  # Board pin 21 (BCM 9)
SPIMOSI = 19  # Board pin 19 (BCM 10)
SPICS = 24    # Board pin 24 (BCM 8)

# photoresistor connected to adc #0
photo_ch = 0

# read SPI data from MCP3008 chip, 8 possible adc's (0 thru 7)
def readadc(adcnum, clockpin, mosipin, misopin, cspin):
    if ((adcnum > 7) or (adcnum < 0)):
        return -1
    GPIO.output(cspin, True)

    GPIO.output(clockpin, False)  # start clock low
    GPIO.output(cspin, False)     # bring CS low

    commandout = adcnum
    commandout |= 0x18  # start bit + single-ended bit
    commandout <<= 3    # we only need to send 5 bits here

    for i in range(5):
        if (commandout & 0x80):
            GPIO.output(mosipin, True)
        else:
            GPIO.output(mosipin, False)
        commandout <<= 1
        GPIO.output(clockpin, True)
        GPIO.output(clockpin, False)

    adcout = 0
    # read in one empty bit, one null bit and 10 ADC bits
    for i in range(12):
        GPIO.output(clockpin, True)
        GPIO.output(clockpin, False)
        adcout <<= 1
        if (GPIO.input(misopin)):
            adcout |= 0x1

    GPIO.output(cspin, True)
    adcout >>= 1  # first bit is 'null' so drop it
    return adcout

```

```python
def main():
    try:
        # Use BOARD mode (physical pin numbers)
        GPIO.setmode(GPIO.BOARD)
        GPIO.setwarnings(False)

        # Setup SPI pins
        GPIO.setup(SPIMOSI, GPIO.OUT)
        GPIO.setup(SPIMISO, GPIO.IN)
        GPIO.setup(SPICLK, GPIO.OUT)
        GPIO.setup(SPICS, GPIO.OUT)

        # Read ADC value once
        value = readadc(photo_ch, SPICLK, SPIMOSI, SPIMISO, SPICS)
        print(value)  # Print value for C++ to read

    except Exception as e:
        print(f"Error: {e}", file=sys.stderr)
        print(0)  # Return 0 on error
    finally:
        GPIO.cleanup()

if __name__ == '__main__':
    main()
```

Lab10.cpp(有點長, 用文字敘述):

```
#define _GNU_SOURCE  // 需要這個才能使用 pthread_tryjoin_np

#include <iostream>

#include <pthread.h>

#include <unistd.h>

#include <fcntl.h>

#include <string.h>

#include <stdio.h>

#include <stdlib.h>

#include <sys/socket.h>

#include <netinet/in.h>


using namespace std;


#define DEVICE_PATH "/dev/demo"


// ==========================================

// Shared Data (Global Variables)

// ==========================================

int current_adc_value = 0;      // ADC value from photoresistor

bool is_system_running = false; // System Start/Stop flag
(controlled by Web)

int threshold = 500;            // Light threshold (ADC < 500 =
bright enough)

int fd;                         // Driver file descriptor


// Computation statistics (Lab9 style - Data Parallelism)
```

```
long long total_computations = 0;        // Total number of
computations completed

long long last_sum_result = 0;           // Last summation result
(1 to 50,000,000)

long long last_t1_result = 0;            // T1 result (1 to
25,000,000)

long long last_t2_result = 0;            // T2 result (25,000,001
to 50,000,000)

bool is_computing = false;               // Current computation
status

bool is_paused = false;                  // 運算是否被暫停

double computation_time = 0.0;           // Last computation time
in seconds

int led_blink_speed = 2;                 // LED blink speed
(seconds)


// 即時運算進度（用於 Web 顯示）

volatile long long live_t1_index = 0;    // T1 目前計算到的索引

volatile long long live_t1_sum = 0;      // T1 目前的部分和

volatile long long live_t2_index = 0;    // T2 目前計算到的索引

volatile long long live_t2_sum = 0;      // T2 目前的部分和


// ==========================================

// 暫停/恢復控制變數

// ==========================================

volatile bool pause_computation = false;   // 暫停旗標

pthread_mutex_t pause_mutex = PTHREAD_MUTEX_INITIALIZER;

pthread_cond_t pause_cond = PTHREAD_COND_INITIALIZER;
```

```
// 保存暫停時的運算狀態
struct ComputationState {
    long long t1_current_index;    // T1 目前計算到的位置
    long long t1_partial_sum;      // T1 目前的部分和
    long long t2_current_index;    // T2 目前計算到的位置
    long long t2_partial_sum;      // T2 目前的部分和
    bool has_saved_state;          // 是否有保存的狀態
    double elapsed_before_pause;   // 暫停前已經過的時間
};

ComputationState saved_state = {0, 0, 0, 0, false, 0.0};

// Mutex to protect shared resources
pthread_mutex_t data_mutex = PTHREAD_MUTEX_INITIALIZER;

// ==========================================
// Helper Function: Control LED via Driver
// ==========================================
void control_led(const char* cmd) {
    if (fd >= 0) {
        write(fd, cmd, strlen(cmd));
    }
}

// ==========================================
// Thread A: Sensor Node
// Purpose: Read ADC value from Python script
```

```
// ==========================================

void* thread_sensor(void* arg) {

    char buffer[128];


    while (1) {

        // Call Python script to read ADC value

        FILE* pipe = popen("python3 read_adc.py", "r");

        if (pipe) {

            if (fgets(buffer, sizeof(buffer), pipe) != NULL) {

                int val = atoi(buffer);


                // Critical Section: Update shared data with
mutex

                pthread_mutex_lock(&data_mutex);

                current_adc_value = val;

                pthread_mutex_unlock(&data_mutex);


                printf("[Sensor] ADC Value: %d\n", val);

            }

            pclose(pipe);

        } else {

            printf("[Sensor] Error: Failed to execute Python
script\n");

        }


        usleep(500000); // Read every 0.5 seconds

    }
```

```
    pthread_exit(NULL);
}

// ==========================================
// Helper Function: Worker computation (Data Parallelism) - 支援暫停/恢復
// ==========================================
struct WorkerArgs {
    long long start;
    long long end;
    long long* result;
    long long* current_index;      // 用於保存目前進度
    long long initial_sum;         // 從這個值開始累加（用於恢復）
    long long resume_from;         // 從這個索引開始（用於恢復）
    volatile long long* live_index;  // 指向全域即時索引
    volatile long long* live_sum;    // 指向全域即時部分和
};

void* worker_computation(void* arg) {
    WorkerArgs* args = (WorkerArgs*)arg;
    long long sum = args->initial_sum;  // 從保存的部分和開始

    int check_interval = 50000;  // 每 50,000 次迭代檢查一次暫停旗標
    int delay_us = 20000;        // 每次延遲 20ms

    // 從 resume_from 開始（正常情況是 start，恢復時是上次暫停的位置）
    long long start_from = (args->resume_from > args->start) ? args->resume_from : args->start;
```

```
    for (long long i = start_from; i <= args->end; i++) {
        sum += i;

        // 只在特定間隔檢查暫停旗標（避免每次迭代都 lock/unlock）
        if ((i - start_from) % check_interval == 0) {
            // 更新目前索引（用於顯示進度）
            *(args->current_index) = i;
            *(args->result) = sum;

            // 更新即時進度到全域變數（供 Web 顯示）
            *(args->live_index) = i;
            *(args->live_sum) = sum;

            // 檢查暫停旗標
            pthread_mutex_lock(&pause_mutex);
            while (pause_computation) {
                printf("[Worker Thread] 暫停於 i=%lld, 部分和
=%lld\n", i, sum);

                // 等待恢復訊號
                pthread_cond_wait(&pause_cond, &pause_mutex);
                printf("[Worker Thread] 恢復運算從 i=%lld\n", i);
            }
            pthread_mutex_unlock(&pause_mutex);

            // 加入延遲讓運算變慢（約 10 秒完成）
```

```
            if (i != start_from) {
                usleep(delay_us);
            }
        }
    }

    *(args->result) = sum;
    *(args->current_index) = args->end + 1;  // 表示完成
    *(args->live_index) = args->end;
    *(args->live_sum) = sum;
    return NULL;
}

// ==========================================
// Thread B: Worker Node
// Purpose: Perform heavy computation (Lab9 Data Parallelism) and control LED
// ==========================================
void* thread_worker(void* arg) {
    while (1) {
        int adc;
        bool run;

        // Critical Section: Read shared data with mutex
        pthread_mutex_lock(&data_mutex);
        adc = current_adc_value;
        run = is_system_running;
```

```
        pthread_mutex_unlock(&data_mutex);


        // Decision Logic
        if (run && adc < threshold) {
            // Condition 1: System ON + Bright enough → Lab9 Data
Parallelism + Fast LED blink
            printf("[Worker] System ON, Bright (ADC=%d) →
Starting Lab9 Computation...\n", adc);


            // Update computing status
            pthread_mutex_lock(&data_mutex);
            is_computing = true;
            is_paused = false;
            led_blink_speed = 0.5;  // Fast blink (0.5s)
            pthread_mutex_unlock(&data_mutex);


            // === LAB9 DATA PARALLELISM: Sum from 1 to
50,000,000 ===


            // Record start time
            struct timespec start_time, end_time;
            clock_gettime(CLOCK_MONOTONIC, &start_time);


            double elapsed_offset = 0.0;  // 如果是恢復運算，加上之前
已經過的時間


            pthread_t t1, t2;
            long long result_t1 = 0, result_t2 = 0;
```

```
            long long current_index_t1 = 0, current_index_t2 = 0;

            // 重設暫停旗標
            pthread_mutex_lock(&pause_mutex);
            pause_computation = false;
            pthread_mutex_unlock(&pause_mutex);

            // 重設即時進度變數
            live_t1_index = 0;
            live_t1_sum = 0;
            live_t2_index = 25000001;
            live_t2_sum = 0;

            // 檢查是否有保存的狀態需要恢復
            WorkerArgs args_t1, args_t2;

            if (saved_state.has_saved_state) {
                printf("[Worker] 🔄 恢復之前的運算狀態...\n");
                printf("         T1: 從 index=%lld, sum=%lld 繼續
\n",
                       saved_state.t1_current_index,
saved_state.t1_partial_sum);
                printf("         T2: 從 index=%lld, sum=%lld 繼續
\n",
                       saved_state.t2_current_index,
saved_state.t2_partial_sum);

                // 恢復 T1 狀態
```

```
                args_t1 = {1, 25000000, &result_t1,
&current_index_t1,
                           saved_state.t1_partial_sum,
saved_state.t1_current_index,
                           &live_t1_index, &live_t1_sum};


                // 恢復 T2 狀態
                args_t2 = {25000001, 50000000, &result_t2,
&current_index_t2,
                           saved_state.t2_partial_sum,
saved_state.t2_current_index,
                           &live_t2_index, &live_t2_sum};


                // 設定恢復時的即時進度
                live_t1_index = saved_state.t1_current_index;
                live_t1_sum = saved_state.t1_partial_sum;
                live_t2_index = saved_state.t2_current_index;
                live_t2_sum = saved_state.t2_partial_sum;


                elapsed_offset =
saved_state.elapsed_before_pause;
                saved_state.has_saved_state = false;  // 清除保存的
狀態
            } else {
                // 全新開始
                args_t1 = {1, 25000000, &result_t1,
&current_index_t1, 0, 1,
                           &live_t1_index, &live_t1_sum};
                args_t2 = {25000001, 50000000, &result_t2,
&current_index_t2, 0, 25000001,
```

```
                        &live_t2_index, &live_t2_sum};
            }

            // Create worker threads
            pthread_create(&t1, NULL, worker_computation,
&args_t1);
            pthread_create(&t2, NULL, worker_computation,
&args_t2);

            printf("[Worker] 運算開始，預計需要約 10 秒...\n");

            bool t1_done = false, t2_done = false;
            bool was_paused = false;
            int blink_count = 0;

            while (!t1_done || !t2_done) {
                // === 檢查光線狀態和系統狀態 ===
                int current_adc;
                bool current_run;
                pthread_mutex_lock(&data_mutex);
                current_adc = current_adc_value;
                current_run = is_system_running;
                pthread_mutex_unlock(&data_mutex);

                // 如果光線過暗或系統停止，發送暫停訊號
                if (current_adc >= threshold || !current_run) {
                    pthread_mutex_lock(&pause_mutex);
                    if (!pause_computation) {
```

```
                        pause_computation = true;
                        was_paused = true;

                        if (!current_run) {
                            printf("[Worker] ⏸️ 系統已停止！運算暫停中...\n");
                        } else {
                            printf("[Worker] ⏸️ 光線過暗 (ADC=%d >= %d)！運算暫停中...\n",
                                   current_adc, threshold);
                        }
                    }
                    pthread_mutex_unlock(&pause_mutex);

                    // 更新暫停狀態（在 pause_mutex 外面）
                    pthread_mutex_lock(&data_mutex);
                    is_paused = true;
                    pthread_mutex_unlock(&data_mutex);

                    // 暫停時使用慢速 LED 閃爍
                    control_led("LED1on");
                    usleep(500000); // 0.5s ON
                    control_led("LED1off");
                    usleep(500000); // 0.5s OFF

                } else {
                    // 光線恢復，發送恢復訊號
                    pthread_mutex_lock(&pause_mutex);
```

```
                    if (pause_computation) {
                        pause_computation = false;
                        pthread_cond_broadcast(&pause_cond);  //
喚醒所有等待的執行緒
                        printf("[Worker] ▶️ 光線恢復 (ADC=%d)！繼續
運算...\n", current_adc);
                    }
                    pthread_mutex_unlock(&pause_mutex);

                    // 更新暫停狀態（在 pause_mutex 外面）
                    pthread_mutex_lock(&data_mutex);
                    is_paused = false;
                    pthread_mutex_unlock(&data_mutex);

                    // 正常運算時 LED 快速閃爍
                    control_led("LED1on");
                    usleep(250000); // 0.25s ON
                    control_led("LED1off");
                    usleep(250000); // 0.25s OFF
                    blink_count++;
                }

                // 檢查執行緒是否完成 (使用 non-blocking join)
                if (!t1_done) {
                    int ret = pthread_tryjoin_np(t1, NULL);
                    if (ret == 0) {
                        t1_done = true;
```

```
                        printf("[Worker] T1 完成! 結果=%lld\n",
result_t1);
                    }
                }
                if (!t2_done) {
                    int ret = pthread_tryjoin_np(t2, NULL);
                    if (ret == 0) {
                        t2_done = true;
                        printf("[Worker] T2 完成! 結果=%lld\n",
result_t2);
                    }
                }

                // 顯示進度（每 4 次閃爍）
                if (blink_count % 4 == 0 && blink_count > 0) {
                    double progress_t1 = (current_index_t1 > 0) ?
                        (double)(current_index_t1 - 1) /
25000000.0 * 100.0 : 0;
                    double progress_t2 = (current_index_t2 >
25000001) ?
                        (double)(current_index_t2 - 25000001) /
25000000.0 * 100.0 : 0;
                    printf("[Worker] 運算進度: T1=%.1f%%,
T2=%.1f%%\n", progress_t1, progress_t2);
                }
            }

            // 運算完成
            // Calculate final result
```

```
            long long final_sum = result_t1 + result_t2;


            // Record end time

            clock_gettime(CLOCK_MONOTONIC, &end_time);

            double elapsed = elapsed_offset + (end_time.tv_sec -
start_time.tv_sec) +

                            (end_time.tv_nsec -
start_time.tv_nsec) / 1e9;


            // Update shared computation results

            pthread_mutex_lock(&data_mutex);

            total_computations++;

            last_sum_result = final_sum;

            last_t1_result = result_t1;

            last_t2_result = result_t2;

            computation_time = elapsed;

            is_computing = false;

            is_paused = false;

            is_system_running = false;  // 運算完成後自動停止系統

            pthread_mutex_unlock(&data_mutex);


            // 清除保存的狀態

            saved_state.has_saved_state = false;


            printf("[Worker] ✓ 運算 #%lld 完成!\n",
total_computations);

            printf("         T1 (1~25M): %lld\n", result_t1);

            printf("         T2 (25M~50M): %lld\n", result_t2);
```

```
            printf("          總和: %lld\n", final_sum);
            printf("          耗時: %.3f 秒\n", elapsed);
            printf("[Worker] 🛑 系統已自動停止, 等待手動重新啟動...
\n");


            // Turn LED solid ON to indicate completion
            control_led("LED1on");
            usleep(1000000); // Keep ON for 1 second
            control_led("LED1off");


        } else {
            // Condition 2: System OFF or Too Dark → Wait + Slow
LED blink
            pthread_mutex_lock(&data_mutex);
            is_computing = false;
            led_blink_speed = 2;  // Slow blink (2s)
            pthread_mutex_unlock(&data_mutex);


            if (!run) {
                if (saved_state.has_saved_state) {
                    printf("[Worker] 系統已停止 (有保存的運算狀態, 重啟
後將繼續)\n");
                } else {
                    printf("[Worker] 系統已停止 → 待命中...\n");
                }
            } else {
                printf("[Worker] 光線過暗 (ADC=%d) → 待命中...\n",
adc);
```

```
            }

            // Slow LED blink (1s ON, 1s OFF)
            control_led("LED1on");
            usleep(1000000); // 1s ON
            control_led("LED1off");
            usleep(1000000); // 1s OFF
        }
    }

    pthread_exit(NULL);
}

// ==========================================
// Thread C: Web Server (GUI)
// Purpose: Provide monitoring interface
// ==========================================
void* thread_web(void* arg) {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    int opt = 1;
    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt,
sizeof(opt));
```

```
    address.sin_family = AF_INET;

    address.sin_addr.s_addr = INADDR_ANY;

    address.sin_port = htons(8080);


    bind(server_fd, (struct sockaddr *)&address,
sizeof(address));

    listen(server_fd, 3);


    printf("[Web] Server started on port 8080\n");


    while (1) {

        new_socket = accept(server_fd, (struct sockaddr
*)&address, (socklen_t*)&addrlen);


        char buffer[1024] = {0};

        read(new_socket, buffer, 1024);


        // Toggle system running state

        if (strstr(buffer, "GET /toggle") != NULL) {

            pthread_mutex_lock(&data_mutex);

            is_system_running = !is_system_running;

            pthread_mutex_unlock(&data_mutex);


            // Redirect back to main page

            string redirect = "HTTP/1.1 302 Found\r\nLocation:
/\r\n\r\n";
```

```
            send(new_socket, redirect.c_str(), redirect.length(),
0);

            close(new_socket);

            continue;

        }


        // Get current values with mutex protection

        int adc_display;

        bool run_display, computing_display, paused_display;

        long long total_comp_display, sum_result_display;

        long long t1_result_display, t2_result_display;

        double comp_time_display;

        int blink_speed_display;

        bool has_saved_state_display;


        // 即時運算進度

        long long live_t1_idx, live_t1_s, live_t2_idx, live_t2_s;


        pthread_mutex_lock(&data_mutex);

        adc_display = current_adc_value;

        run_display = is_system_running;

        computing_display = is_computing;

        paused_display = is_paused;

        total_comp_display = total_computations;

        sum_result_display = last_sum_result;

        t1_result_display = last_t1_result;

        t2_result_display = last_t2_result;
```

```
        comp_time_display = computation_time;

        blink_speed_display = led_blink_speed;

        has_saved_state_display = saved_state.has_saved_state;

        pthread_mutex_unlock(&data_mutex);


        // 讀取即時運算進度（這些是 volatile，不需要 mutex）

        live_t1_idx = live_t1_index;

        live_t1_s = live_t1_sum;

        live_t2_idx = live_t2_index;

        live_t2_s = live_t2_sum;


        // Build HTML response

        string html = "HTTP/1.1 200 OK\r\nContent-Type:
text/html; charset=utf-8\r\n\r\n";

        html += "<!DOCTYPE html><html><head>";

        html += "<meta charset='UTF-8'>";

        html += "<meta http-equiv='refresh' content='1'>";

        html += "<title>IoT 智慧運算監控系統</title>";

        html += "<style>";

        html += "body { font-family: 'Segoe UI', Arial,
sans-serif; background: linear-gradient(135deg, #1a1a2e 0%,
#16213e 100%); color: white; margin: 0; padding: 20px;
min-height: 100vh; }";

        html += ".container { max-width: 900px; margin: 0 auto;
}";

        html += "h1 { text-align: center; color: #00d4ff;
text-shadow: 0 0 20px rgba(0,212,255,0.5); margin-bottom: 10px;
}";

        html += ".subtitle { text-align: center; color: #888;
margin-bottom: 30px; }";
```

```
        html += ".status-grid { display: grid;
grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap:
20px; margin-bottom: 30px; }";

        html += ".status-card { background:
rgba(255,255,255,0.1); border-radius: 15px; padding: 20px;
text-align: center; backdrop-filter: blur(10px); }";

        html += ".status-card h3 { margin: 0 0 15px 0; color:
#00d4ff; font-size: 14px; }";

        html += ".value { font-size: 28px; font-weight: bold;
margin: 10px 0; }";

        html += ".value.running { color: #00ff88; }";

        html += ".value.stopped { color: #ff6b6b; }";

        html += ".value.computing { color: #00d4ff; }";

        html += ".value.paused { color: #ffa500; }";

        html += ".value.standby { color: #888; }";

        html += ".progress-bar { background:
rgba(255,255,255,0.2); border-radius: 10px; height: 10px;
overflow: hidden; }";

        html += ".progress-fill { background:
linear-gradient(90deg, #00d4ff, #00ff88); height: 100%;
transition: width 0.3s; }";

        html += "button { background: linear-gradient(135deg,
#00d4ff, #00ff88); border: none; padding: 15px 40px; font-size:
18px; border-radius: 25px; cursor: pointer; color: #1a1a2e;
font-weight: bold; transition: transform 0.2s; }";

        html += "button:hover { transform: scale(1.05); }";

        html += ".computation-box { background:
rgba(0,212,255,0.1); border: 1px solid #00d4ff; border-radius:
15px; padding: 20px; margin: 20px 0; }";

        html += ".computation-box h2 { color: #00d4ff;
margin-top: 0; }";

        html += ".comp-stat { display: flex; justify-content:
space-between; padding: 10px 0; border-bottom: 1px solid
rgba(255,255,255,0.1); }";
```

```
        html += ".comp-label { color: #888; }";

        html += ".comp-value { color: #00ff88; font-weight: bold;
}";

        html += ".lab9-section { background: rgba(255,193,7,0.1);
border: 1px solid #ffc107; border-radius: 15px; padding: 20px;
margin: 20px 0; }";

        html += ".lab9-section h2 { color: #ffc107; margin-top:
0; }";

        html += ".worker-detail { background: rgba(0,0,0,0.2);
border-radius: 10px; padding: 15px; margin: 10px 0; }";

        html += ".worker-detail h4 { margin: 0 0 10px 0; color:
#00d4ff; }";

        html += ".led-indicator { display: inline-block; width:
20px; height: 20px; border-radius: 50%; margin: 0 5px; animation:
blink 1s infinite; }";

        html += ".led-fast { background: #28a745; animation:
blink 0.5s infinite; }";

        html += ".led-slow { background: #ffc107; animation:
blink 2s infinite; }";

        html += ".led-paused { background: #ffa500; animation:
blink 1s infinite; }";

        html += "@keyframes blink { 0%, 49% { opacity: 1; } 50%,
100% { opacity: 0.3; } }";

        html += ".pause-banner { background: rgba(255,165,0,0.2);
border: 2px solid #ffa500; border-radius: 10px; padding: 15px;
margin: 20px 0; text-align: center; }";

        html += ".pause-banner h3 { color: #ffa500; margin: 0;
}";

        html += "</style>";

        html += "</head><body>";


        html += "<div class='container'>";
```

```
        html += "<h1>🌟 IoT 智慧運算監控系統 🌟</h1>";
        html += "<div class='subtitle'>Lab10: 環境感知運算控制 (整合
Lab9 資料平行運算)</div>";

        // 暫停警告橫幅
        if (paused_display) {
            html += "<div class='pause-banner'>";
            html += "<h3>⏸️ 運算已暫停 - 等待光線恢復...</h3>";
            html += "<p>運算進度已保存，光線充足時將自動繼續</p>";
            html += "</div>";
        }

        // 有保存狀態的提示
        if (has_saved_state_display && !computing_display) {
            html += "<div class='pause-banner'>";
            html += "<h3>💾 有未完成的運算</h3>";
            html += "<p>啟動系統後將從上次暫停處繼續運算</p>";
            html += "</div>";
        }

        // Status Grid
        html += "<div class='status-grid'>";

        // ADC Value Card
        html += "<div class='status-card'>";
        html += "<h3>💡 光感測數值</h3>";
        html += "<div class='value'>" + to_string(adc_display) +
"</div>";
```

```
        html += "<div class='progress-bar'><div
class='progress-fill' style='width: " + to_string((adc_display *
100) / 1023) + "%'></div></div>";

        html += "</div>";


        // System Status Card

        html += "<div class='status-card'>";

        html += "<h3>🔧 系統狀態</h3>";

        html += "<div class='value " + string(run_display ?
"running" : "stopped") + "'>";

        html += run_display ? "⚡ 運行中" : "⏸️ 已停止";

        html += "</div>";

        html += "</div>";


        // Environment Card

        html += "<div class='status-card'>";

        html += "<h3>☀️ 環境光線</h3>";

        html += "<div class='value'>";

        if (adc_display < threshold) {

            html += "✓ 充足";

        } else {

            html += "✗ 不足";

        }

        html += "</div>";

        html += "<small>閾值: " + to_string(threshold) +
"</small>";

        html += "</div>";
```

```
        // Worker Status Card
        html += "<div class='status-card'>";
        html += "<h3>⚙️ 運算狀態</h3>";
        string status_class = "standby";
        string status_text = "💤 待命";
        if (paused_display) {
            status_class = "paused";
            status_text = "⏸️ 已暫停";
        } else if (computing_display) {
            status_class = "computing";
            status_text = "🔄 計算中";
        }
        html += "<div class='value " + status_class + "'>" +
status_text + "</div>";

        string led_class = paused_display ? "led-paused" :
(blink_speed_display < 1 ? "led-fast" : "led-slow");
        html += "<span class='led-indicator " + led_class +
"'></span>";
        html += "</div>";

        html += "</div>"; // End status-grid

        // Lab9 Data Parallelism Section
        html += "<div class='lab9-section'>";
        html += "<h2>📊 Lab9 資料平行運算 (Data Parallelism)</h2>";
        html += "<p style='color: #856404; margin: 10px 0;'>計算 1
+ 2 + 3 + ... + 50,000,000 的總和</p>";
```

```
        // 計算進度百分比
        double t1_progress = (live_t1_idx > 0) ?
(double)(live_t1_idx) / 25000000.0 * 100.0 : 0;
        double t2_progress = (live_t2_idx > 25000001) ?
(double)(live_t2_idx - 25000001) / 25000000.0 * 100.0 : 0;
        if (t1_progress > 100) t1_progress = 100;
        if (t2_progress > 100) t2_progress = 100;

        // Worker Details - T1
        html += "<div class='worker-detail'>";
        html += "<h4>🔹 Thread T1 (Worker 1)</h4>";
        html += "<div class='comp-stat'>";
        html += "<span class='comp-label'>計算範圍:</span>";
        html += "<span class='comp-value'>1 ~ 25,000,000</span>";
        html += "</div>";

        // T1 即時進度
        if (computing_display || paused_display) {
            html += "<div class='comp-stat'>";
            html += "<span class='comp-label'>📍 目前位置:</span>";
            html += "<span class='comp-value' style='color:
#00d4ff;'>" + to_string(live_t1_idx) + "</span>";
            html += "</div>";
            html += "<div class='comp-stat'>";
            html += "<span class='comp-label'>📊 即時部分和:
</span>";
            html += "<span class='comp-value' style='color:
#ffa500;'>" + to_string(live_t1_s) + "</span>";
```

```
            html += "</div>";

            // 進度條

            char progress_str[20];

            snprintf(progress_str, sizeof(progress_str), "%.1f",
t1_progress);

            html += "<div class='comp-stat'>";

            html += "<span class='comp-label'>進度:</span>";

            html += "<span class='comp-value'>" +
string(progress_str) + "%</span>";

            html += "</div>";

            html += "<div class='progress-bar'><div
class='progress-fill' style='width: " + string(progress_str) +
"%'></div></div>";

        }


        html += "<div class='comp-stat'>";

        html += "<span class='comp-label'>最終結果:</span>";

        html += "<span class='comp-value'>" +
to_string(t1_result_display) + "</span>";

        html += "</div>";

        html += "</div>";


        // Worker Details - T2

        html += "<div class='worker-detail'>";

        html += "<h4>🔹 Thread T2 (Worker 2)</h4>";

        html += "<div class='comp-stat'>";

        html += "<span class='comp-label'>計算範圍:</span>";

        html += "<span class='comp-value'>25,000,001 ~
50,000,000</span>";
```

```
        html += "</div>";


        // T2 即時進度
        if (computing_display || paused_display) {
            html += "<div class='comp-stat'>";
            html += "<span class='comp-label'>📍 目前位置:</span>";
            html += "<span class='comp-value' style='color:
#00d4ff;'>" + to_string(live_t2_idx) + "</span>";
            html += "</div>";
            html += "<div class='comp-stat'>";
            html += "<span class='comp-label'>📊 即時部分和:
</span>";
            html += "<span class='comp-value' style='color:
#ffa500;'>" + to_string(live_t2_s) + "</span>";
            html += "</div>";
            // 進度條
            char progress_str2[20];
            snprintf(progress_str2, sizeof(progress_str2),
"%.1f", t2_progress);
            html += "<div class='comp-stat'>";
            html += "<span class='comp-label'>進度:</span>";
            html += "<span class='comp-value'>" +
string(progress_str2) + "%</span>";
            html += "</div>";
            html += "<div class='progress-bar'><div
class='progress-fill' style='width: " + string(progress_str2) +
"%'></div></div>";
        }
```

```
        html += "<div class='comp-stat'>";

        html += "<span class='comp-label'>最終結果:</span>";

        html += "<span class='comp-value'>" +
to_string(t2_result_display) + "</span>";

        html += "</div>";

        html += "</div>";


        // 即時總和（運算中顯示）

        if (computing_display || paused_display) {

            html += "<div class='worker-detail'
style='background: rgba(0,255,136,0.1); border: 1px solid
#00ff88;'>";

            html += "<h4>⚡ 即時運算總和 (T1 + T2)</h4>";

            html += "<div class='comp-stat'>";

            html += "<span class='comp-label'>目前總和:</span>";

            html += "<span class='comp-value' style='font-size:
24px; color: #00ff88;'>" + to_string(live_t1_s + live_t2_s) +
"</span>";

            html += "</div>";

            html += "</div>";

        }


        html += "<div class='worker-detail'>";

        html += "<h4>🎯 最終結果 (T1 + T2)</h4>";

        html += "<div class='comp-stat'>";

        html += "<span class='comp-label'>總和:</span>";

        html += "<span class='comp-value' style='font-size: 20px;
color: #28a745;'>" + to_string(sum_result_display) + "</span>";

        html += "</div>";
```

```
        html += "<div class='comp-stat'>";
        html += "<span class='comp-label'>計算時間:</span>";
        html += "<span class='comp-value'>";

        // Format computation time
        char time_str[50];
        snprintf(time_str, sizeof(time_str), "%.3f 秒",
comp_time_display);
        html += string(time_str);
        html += "</span>";
        html += "</div>";
        html += "</div>";

        html += "</div>"; // End lab9-section

        // Computation Statistics Box
        html += "<div class='computation-box'>";
        html += "<h2>📈 運算統計資訊</h2>";

        html += "<div class='comp-stat'>";
        html += "<span class='comp-label'>總運算次數:</span>";
        html += "<span class='comp-value'>" +
to_string(total_comp_display) + " 次</span>";
        html += "</div>";

        html += "<div class='comp-stat'>";
        html += "<span class='comp-label'>運算模式:</span>";
```

```
        html += "<span class='comp-value'>資料平行 (Data
Parallelism)</span>";

        html += "</div>";


        html += "<div class='comp-stat'>";

        html += "<span class='comp-label'>執行緒架構:</span>";

        html += "<span class='comp-value'>2 Worker Threads (T1 +
T2)</span>";

        html += "</div>";


        html += "<div class='comp-stat'>";

        html += "<span class='comp-label'>運算條件:</span>";

        html += "<span class='comp-value'>";

        if (paused_display) {

            html += "⏸ 暫停中 (光線不足)";

        } else if (run_display && adc_display < threshold) {

            html += "✓ 滿足 (系統開啟 且 光線充足)";

        } else if (!run_display) {

            html += "✗ 系統已停止";

        } else {

            html += "✗ 光線不足 (ADC >= " + to_string(threshold) +
")";

        }

        html += "</span>";

        html += "</div>";


        // Expected result info

        html += "<div class='comp-stat'>";
```

```
        html += "<span class='comp-label'>理論值:</span>";

        html += "<span
class='comp-value'>1,250,000,025,000,000</span>";

        html += "</div>";


        html += "</div>"; // End computation-box


        // Control Button

        html += "<a href='/toggle'><button>🔄 切換 啟動/停止
</button></a>";


        html += "<hr style='margin: 30px 0; border: none;
border-top: 1px solid #ddd;'>";

        html += "<p style='color: #888; font-size: 14px;'>⏱️ 自動
更新: 每 1 秒 | 🔗 Port: 8080</p>";

        html += "<p style='color: #666; font-size: 12px;'>💡 提示:
遮住光敏電阻可暫停運算, 移開後自動繼續</p>";


        html += "</div>"; // End container

        html += "</body></html>";


        send(new_socket, html.c_str(), html.length(), 0);

        close(new_socket);

    }


    pthread_exit(NULL);

}


// ==========================================
```

```
// Main Function
// ==========================================
int main() {
    // Open device driver
    fd = open(DEVICE_PATH, O_RDWR);
    if (fd < 0) {
        perror("Failed to open driver");
        printf("Make sure to:\n");
        printf("1. Compile driver: make\n");
        printf("2. Load driver: sudo insmod demo.ko\n");
        printf("3. Create device: sudo mknod /dev/demo c 60
0\n");
        printf("4. Set permission: sudo chmod 666 /dev/demo\n");
        return -1;
    }

    printf("Driver opened successfully!\n");

    pthread_t tA, tB, tC;

    // Create three threads
    if (pthread_create(&tA, NULL, thread_sensor, NULL) != 0) {
        perror("Failed to create Thread A (Sensor)");
        return -1;
    }

    if (pthread_create(&tB, NULL, thread_worker, NULL) != 0) {
```

```
        perror("Failed to create Thread B (Worker)");
        return -1;
    }

    if (pthread_create(&tC, NULL, thread_web, NULL) != 0) {
        perror("Failed to create Thread C (Web)");
        return -1;
    }

    printf("\n=== System Started ===\n");
    printf("Thread A: Sensor monitoring\n");
    printf("Thread B: Worker computing (支援暫停/恢復)\n");
    printf("Thread C: Web server running\n");
    printf("======================\n\n");

    // Wait for threads to complete (they run forever)
    pthread_join(tA, NULL);
    pthread_join(tB, NULL);
    pthread_join(tC, NULL);

    close(fd);
    return 0;
}
```

四、本次實驗過程說明與解決方法

就如同前面所說，我們的實驗過程除了靜態網頁的問題之外沒什麼狀況，但是沒想到我們還是笨手笨腳的，一下子忘記接地，一下子IC接反，到後面終於弄好了才開始做最後的調整，解決方法就是下次還是乖乖寫個靜態網頁的程式，別再靠LLM耍笨。

五、分工

學號、組員	貢獻比例	工作內容
112590034 高翎竣	30%	程式撰寫除錯、報告撰寫
112590032 陳俊彥	30%	程式撰寫除錯、報告撰寫
112590453 詹東儒	40%	程式撰寫除錯、報告撰寫